Baze de Date

Cap. 10. Optimizarea și execuția interogărilor SQL.

Textbook: Ramakrishnan, Gehrke, “Database Management Systems”, McGraw Hill, C15

2023 UPT Conf.Dr. Dan Pescaru

Operatori relaționali

1. O interogare SQL se traduce într-o expresie utilizând operatorii relaționali
2. Expresia se reprezintă ca și un arbore (plan) de execuție care are operatori în noduri și tabele în frunze
3. Atât intrările (frunzele) cât și rezultatul evaluării planului sunt relații (tabele)
4. Schema rezultatului este fixă și se deduce din lista de proiecție

Operatori relaționali

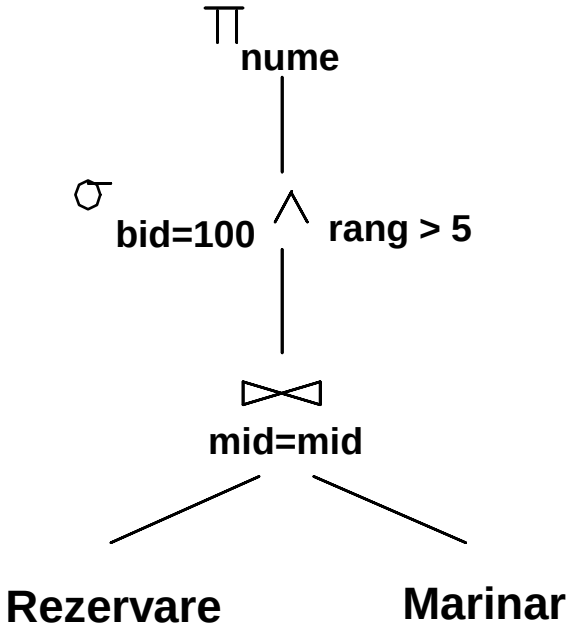
1. Operatori de bază:
   - Proiecție (Π) – selectează coloanele specificate
   - Selecție (σ) – selectează înregistrările specificate
   - Produs Cartezian ($\times$) – combină două relații
   - Diferență mulțimi ($/$) – înregistrările din R1, care nu $\in$ R2
   - Reuniune ($\cup$) – înregistrările din R1 și R2 (fără duplicate)
2. Operatori adiționali:
   - JOIN ($\bullet$), Intersecție ($\cap$) , Divizare ($\div$), Redenumire
3. Din moment ce orice operator returnează o relație, ei pot fi compuși în expresii (Algebra relațională este "închisă")

Evaluarea interogării

1. Plan de execuție = arborele operatorilor din algebră relațională + algoritmii selectați pentru evaluarea lor
2. Două probleme principale în optimizarea interogărilor:
   1. Pentru o interogare, ce planuri sunt considerate?
   2. Care este cel mai bun și cum este estimat costul?
3. Ideal: găsirea cel mai bun plan
4. Practic: evitarea celor mai slabe!

Exemplu de plan de execuție

- Ex. SELECT nume FROM Marinar m
  INNER JOIN Rezervare r ON m.mid=r.mid
  WHERE r.bid=100 AND m.rang>5



Tehnici de optimizare de bază

1. Algoritmii de evaluare a operatorilor relaționali folosesc câteva idei simple
2. Iterația: uneori, mai rapid este a scana toate înregistrările din tabel chiar dacă există un index
3. Uneori, mai rapid este a scana intrările de date într-un index în loc de cele din tabel
   - Indexare: se poate folosi condițiile WHERE pentru a extrage un set mic de înregistrări (σ,°)
   - Partiționare: prin sortare sau hashing, putem partiționa înregistrările de intrare și să înlocuim o operație costisitoare cu operații similare pe intrări mai mici

Catalog și statistici

1. Cataloagele conțin meta-date și statistici:
   - # tupluri (NTuples) și # pagini (NPages) pentru fiecare tabelă
   - # chei distincte (NKeys) și NPages pentru fiecare index
   - Înălțime index, valorile cele mai mici/mari ale cheilor pentru fiecare index arbore B+
2. Cataloage actualizate periodic (pentru eficiență)
3. Actualizarea ori de câte ori se modifică datele este prea costisitoare; oricum sunt multe aproximări, așa că o ușoară inconsecvență este acceptabilă

Estimarea costului

1. Pentru fiecare plan de execuție luat în considerare, trebuie estimat costul fiecărei operații
   - Depinde de cardinalitățile de intrare
   - Depinde de dimensiunea tuplului
2. De asemenea, trebuie să estimeze dimensiunea rezultatului pentru fiecare operator din arbore!
   - Se utilizează informațiile despre relațiile de intrare
   - Pentru selecții și join-uri, se presupune independența predicatelor

Căi de acces

1. O cale de acces este o metodă de extragere a tuplurilor din BD
   - Scanare fișier, sau un index dacă se potrivește cu o selecție (din interogare)
2. Un index arbore B+ se poate folosi cu o conjuncție de termeni care implică atribute într-un prefix al cheii
   - Ex. un index cu cheia <a, b, c> se poate utiliza cu selecția “a=5 AND b=3 și a=5 AND b>6”, dar nu “b=3”
2. Un index hash se potrivește cu o conjuncție de termeni atribut = valoare pentru fiecare atribut din cheia de căutare a indexului
   - Ex. un index Hash cu <a, b, c> se potrivește cu a=5 și b=3 și c=5; dar nu se potrivește cu b=3 sau a>5 și b=3 și c=5

Optimizarea SELECȚIEI

1. Se caută calea de acces cea mai selectivă, se extrag înregistrările folosind-o și apoi se aplică termenii rămași care nu se găsesc în index:
   - Calea de acces cea mai selectivă: un index sau o scanare a fișierelor despre care estimăm că va necesita cele mai puține operații I/O de accesare pagini pe disc
   - Termenii care se potrivesc cu indexul reduc numărul de tupluri extrase; ceilalți termeni sunt folosiți apoi pentru a elimina unele din înregistrările extrase, dar aceasta nu afectează numărul de înregistrări/pagini extrase de pe disc

Optimizarea SELECȚIEI - exemplu

1. Se consideră condiția

   zi<(1/9/23) AND bid=5 AND mid=31

2. Un index arbore B+ cu cheia <zi> poate fi utilizat
   - bid=5 AND mid=31 trebuie apoi verificate pentru fiecare înregistrare extrasă
3. Similar se poate folosi un index hash cu cheia <bid, mid>
   - Condiția zi<(1/9/23) trebuie apoi verificată pentru fiecare înregistrare extrasă

Optimizarea PROIECȚIEI

1. Partea cea mai costisitoare este înlăturarea duplicatelor:
   - Din cauza aceasta interpretoarele de SQL nu înlătură duplicatele decât dacă se specifică DISTINCT în interogare
   - Se poate implementa prin sortare. Poate fi optimizată și să extragă doar câmpurile proiectate în timpul sortării
   - Se poate implementa și prin hash. Se creează partiții folosind o funcție de hash. Se încarcă fiecare partiție în memorie creându-se încă un nivel de partiții pentru a se testa duplicatele

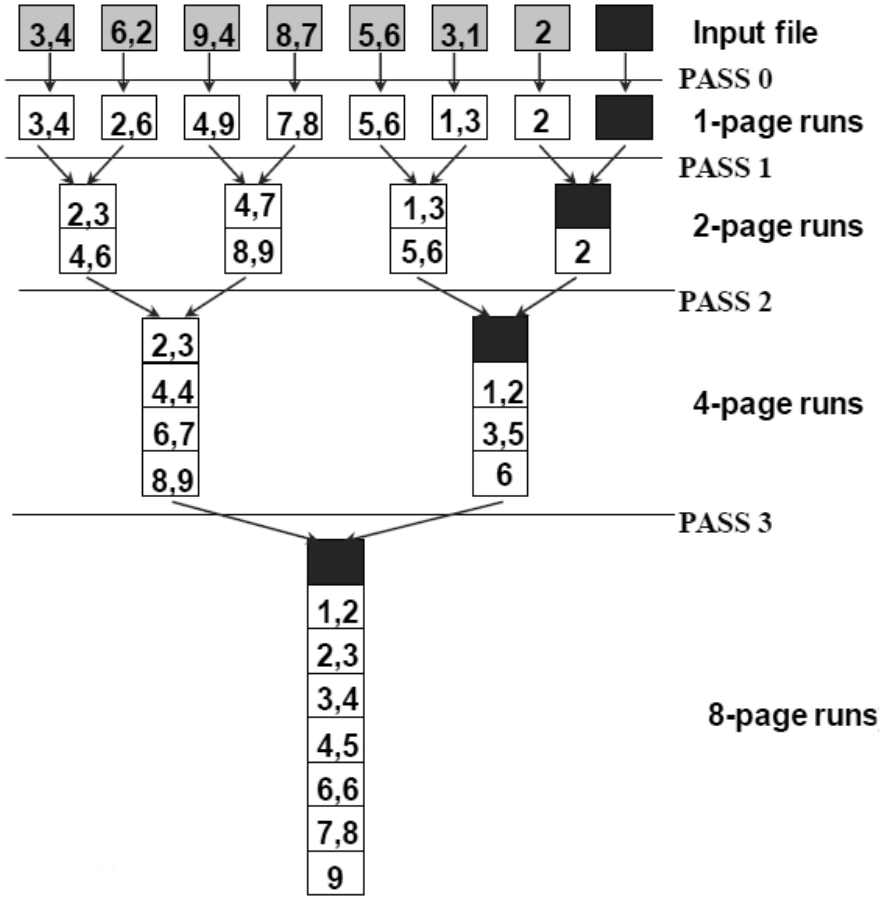
Optimizarea PROIECȚIEI prin sortare

SELECT DISTINCT mid, bid FROM Rezervare

1. O rezolvare prin sortare:
   - Sortare-îmbinare (merge sort) externă pe două căi cu eliminarea câmpurilor care nu sunt în lisat de proiecție
   - Tuplurile din rezultat sunt mai mici decât cele de intrare (rația depinde de nr și dimensiunea câmpurilor care sunt eliminate)
   - Se va modifica pasul de îmbinare pentru a elimina duplicatele rezultând un număr de tupluri mai mic decât la intrare cu nr. de duplicate

Algoritmul de sortare-îmbinare extrenă

1. Idea de bază: sortarea și îmbinarea fișierelor partiție (utilizând divide and conquer), prin trei buffere
2. La fiecare pas: citește + scrie fiecare pagină
3. Cost ~ $2N \times \log_2 N$



Optimizarea PROIECȚIEI prin hashing

SELECT DISTINCT mid, bid FROM Rezervare

1. Faza de partiționare utilizând B buffere: se citește R utilizând un buffer. Pentru fiecare tuplu, fără atribute neproiectate, se aplică funcția hash *h1* pentru a alege unul dintre cele B-1 buffere de ieșire
   - Rezultă B-1 partiții (cu tupluri doar cu atribute proiectate). Două tupluri din partiții diferite nu sunt sigur duplicate
2. Eliminarea duplicatelor: pentru fiecare partiție se creează o tabelă hash în memorie utilizând o funcție hash *h2* (<> *h1*) pe toate câmpurile, iar duplicatele se elimină prin testarea egalității funcției hash

Optimizarea JOIN prin Index Nested Loop

SELECT r.*, m.* FROM Rezervare r, Marinar m
WHERE r.mid=m.mid

1. Fără index: poate să dureze ore de execuție pentru >1M rezervări și câteva mii de marinari!
2. Dacă există un index după coloana de join pentru o tabelă, se face folosește pentru căutare (ciclul intern)
   - Cost: $NrÎnreg_{R1}$ x cost căutare corespondență în R2
3. Pentru fiecare tuplu din R1, costul căutării în indexul R2 este ~ 1.2 pentru hash (< 5 min), și 2-4 pentru index B+ (8-15 min). Costul depinde și de clusterizare
4. Index clusterizat: + 0 operații de I/O
5. Neclusterizat: + 1 operație de I/O per tuplu găsit

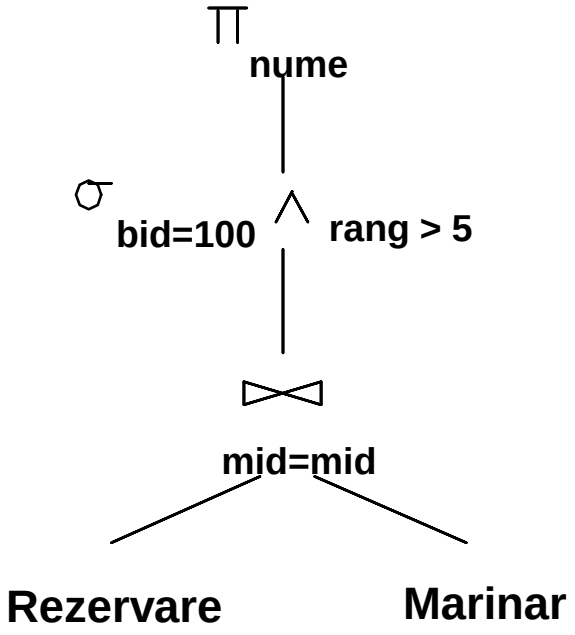
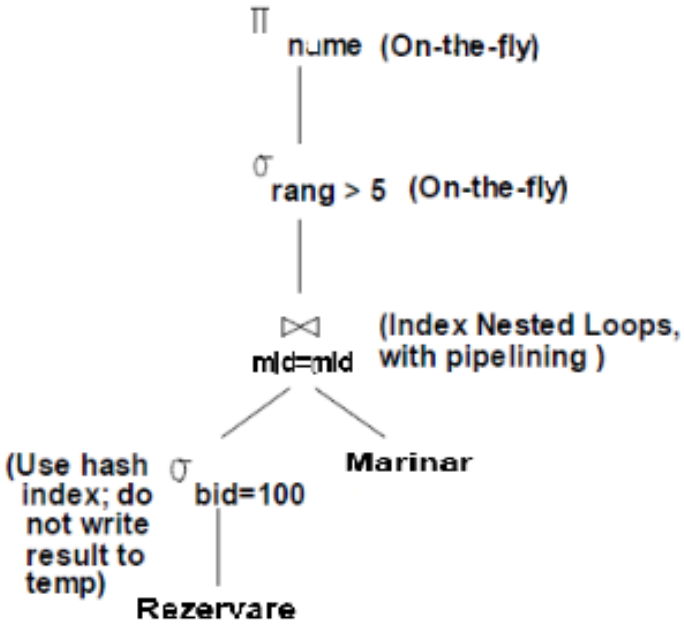
Optimizarea operatorilor pe mulțimi

1. Intersecția și produsul cartezian sunt asemănătoare cu join
2. Reuniunea prin sortare:
   - Se sortează ambele relații (după combinația tuturor atributelor)
   - Îmbinarea relațiilor sortate (algoritm fermoar)
3. Reuniunea prin hash:
   - Se partiționează R1 și R2 cu o funcție hash *h1*
   - Pentru fiecare partiție a lui R2 se va crea o tabelă de hash în memorie (utilizând o funcție *h2*), se scanează partițiile corelate din R1 și se salvează tuplurile eliminându-se și duplicatele

Optimizarea agregării

1. Fără grupare:
   - În general necesită scanarea tabelelor
   - Dacă există un index a cărui cheie conține toate atributele referite în SELECT și WHERE, se poate scana doar indexul fără a se accesa tabela

Optimizarea agregării cu grupare

1. Cu grupare:
   - Se sortează în funcție de atributele grupului, apoi se scanează relația și se calculează funcția de agregare pentru fiecare grup. (Se poate îmbunătăți acest lucru combinând sortarea și calculul agregării)
   - Abordare similară bazată pe hashing pe atributele de grupare
   - Dacă există un index B+cu cheia care să includă toate atributele din SELECT, WHERE și GROUP BY, se poate scana doar indexul; dacă atributele de grupare sunt în prefixul cheii indexului, se pot extrage tuplurile în ordinea grupării și aplica HAVING în același timp

Optimizare - exemplu